Aula 9 - Reutilização de Classes: Sobreposição e Polimorfismo

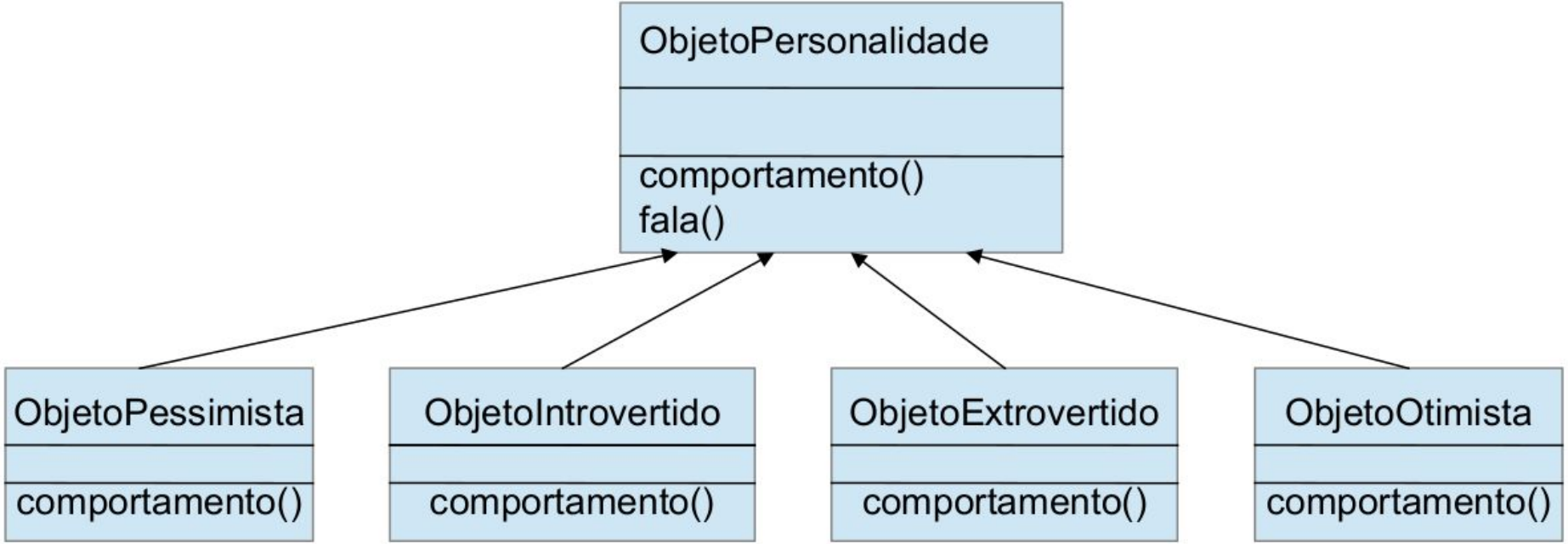
Programação Orientada a Objetos
Prof. Matheus Avila Moreira de Paula
matheusavila@cefetmg.br

Engenharia de Computação
Centro Federal de Educação Tecnológica Leopoldina

Polimorfismo

- Podemos aplicar o conceito de sobrecarga de métodos dentro das classes. Este conceito recebe o nome de polimorfismo
- Em OO, polimorfismo é uma propriedade segundo a qual pode existir, em um programa, vários métodos com o mesmo nome operando em classes diferentes (ou mesmo dentro de uma mesma classe).

Polimorfismo



Polimorfismo

```
public class ObjetoPersonalidade{

    public String comportamento()  {
        return "Eu sou um objeto.";
    }

    public void fale()  {
        System.out.println(comportamento());
    }
}
```

```
public class ObjetoExtrovertido extends
ObjetoPersonalidade
{
    public String comportamento()
    {
        return "E aí!!! Vamos agitar...";
    }
}
```

```
public class ObjetoPessimista extends ObjetoPersonalidade
{
    public String comportamento() {
        return "Nada é tão ruim que não possa piorar.";
    }
}
```

```
public class ObjetoOtimista extends
ObjetoPersonalidade
{
    public String comportamento()
    {
        return "Amanhã sempre
                 será um dia melhor.";
    }
}
```

```
public class ObjetoIntrovertido extends ObjetoPersonalidade
{
    public String comportamento() {
        return "...";
    }
}
```

Polimorfismo

```
public class ExecutaObjetosComPersonalidade {

    public static void main(String[] args) {
        ObjetoPersonalidade personalidade = new ObjetoPersonalidade();
        ObjetoPessimista pessimista       = new ObjetoPessimista();
        ObjetoOtimista otimista           = new ObjetoOtimista();
        ObjetoIntrovertido introvertido   = new ObjetoIntrovertido();
        ObjetoExtrovertido extrovertido   = new ObjetoExtrovertido();

        ObjetoPersonalidade[] personalidades = { personalidade, pessimista,
            otimista, introvertido, extrovertido};

        for(int i=0; i<5; i++) {
            ObjetoPersonalidade persona = personalidades[i];
            persona.fale();
        }
    }
}
```

Polimorfismo

Eu sou um objeto.
Nada é tão ruim que não possa piorar.
Amanhã sempre será um dia melhor.
...
E aí!!! Vamos agitar...

Polimorfismo

- Sobrecarga
  - Métodos de nomes iguais, mas assinaturas diferentes
  - São métodos diferentes, que apenas possuem o mesmo nome
- Sobrescrever
  - Métodos de assinaturas iguais
  - É o mesmo método (em termos de assinatura)
    - Mas podem possuir implementações diferentes
- O que usar com polimorfismo?
  - Ambos podem ser utilizados em termos de polimorfismo

Polimorfismo - Sobrecarga

```java
public class Classe1 {

    private int resultado1;

    protected int calcular(int valor){
        resultado1 = valor * 2;

        return resultado1;
    }
}
```

```java
public class Classe2 extends Classe1{

    private float resultado2;

    protected float calcular(float valor){
        resultado2 = valor * 10;

        return resultado2;
    }
}
```

Polimorfismo - Sobrescrever

```java
public class Classe1 {

    private int resultado1;

    public int calcular(int valor){
        resultado1 = valor * 2;

        return resultado1;
    }
}
```

```java
public class Classe2 extends Classe1{

    private int resultado2;

    public int calcular(int valor){
        resultado2 = valor * 10;

        return resultado2;
    }
}
```

- Polimorfismo
  - De herança – sobrescrita
    - Métodos com mesma assinatura em classes diferentes
    - Entende-se que, neste caso, suas implementações serão diferentes
    - OBS: métodos de mesma assinatura em uma mesma classe não é permitido
    - Posso ter métodos idênticos em classes diferentes?
    - Posso ter métodos idênticos em uma mesma classe?
    - Posso ter métodos idênticos em uma relação de herança (super/subclasse)?

Exercício

- Faça uma classe Animal com atributo idade e um método “som” para retornar “Som generico”
- Faça as classes Homem, com atributo cpf, Cachorro, com atributo raça, e Gato, com atributo numeroDeCores, herdando de animal, criando o método “som” para retornar “Oi” na classe Homem, o método “som” para retornar “Au au” na classe Cachorro e o método “som” para retornar “Miau” na classe Gato
- Crie um vetor de 10 Animais e instancie Homens, Cães e Gatos nesse vetor
- Faça um loop por todos os animais do vetor, chamando o método som de cada um

Exercício

- Faça a sobrescrita do método toString() para imprimir os dados de cada uma das classes do exercício anterior